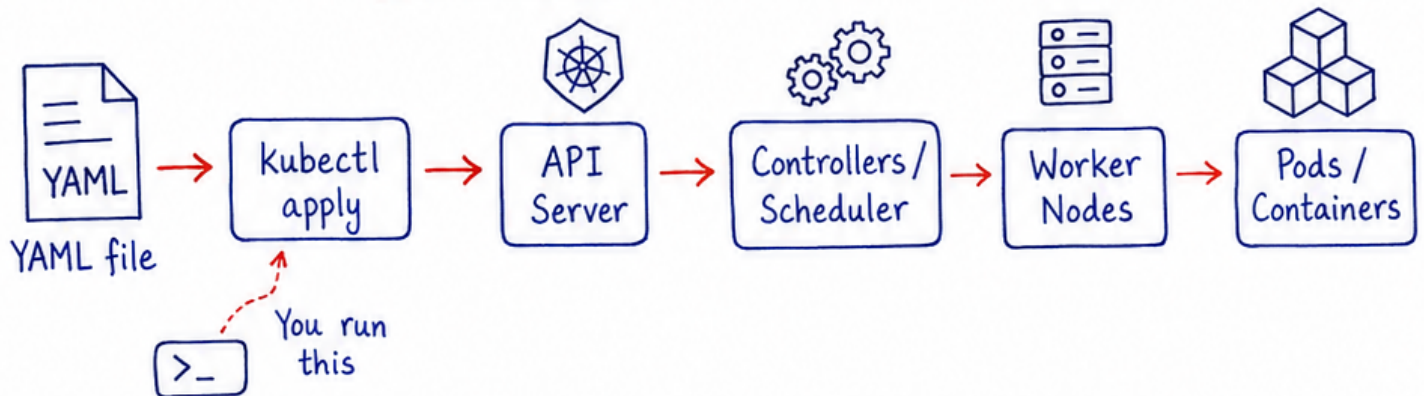


Kubernetes YAML – End-to-End Explanation

What is Kubernetes YAML?

Jyothi Mulkuntla

Kubernetes YAML is a declarative configuration language used to describe the desired state of resources in a Kubernetes cluster.



★ You describe WHAT you want.
Kubernetes decides HOW to maintain it. 💡

Basic YAML Structure

apiVersion	→	apiVersion = which Kubernetes API
kind	→	kind = what resource
metadata	→	metadata = object identity
spec	→	spec = desired configuration

Every Kubernetes object follows this structure.

Common apiVersion examples

- **v1** → Pod, Service
- **apps/v1** → Deployment
- **networking.k8s.io/v1** → Ingress

apiVersion tells Kubernetes which API group to use.

kind examples

- | | |
|---|---|
| <ul style="list-style-type: none">• Pod• Deployment• Service• ConfigMap• Secret• Namespace | <ul style="list-style-type: none">• Ingress• StatefulSet• DaemonSet• Job• CronJob |
|---|---|

Metadata, Labels & Spec

JyothiMulkuntla

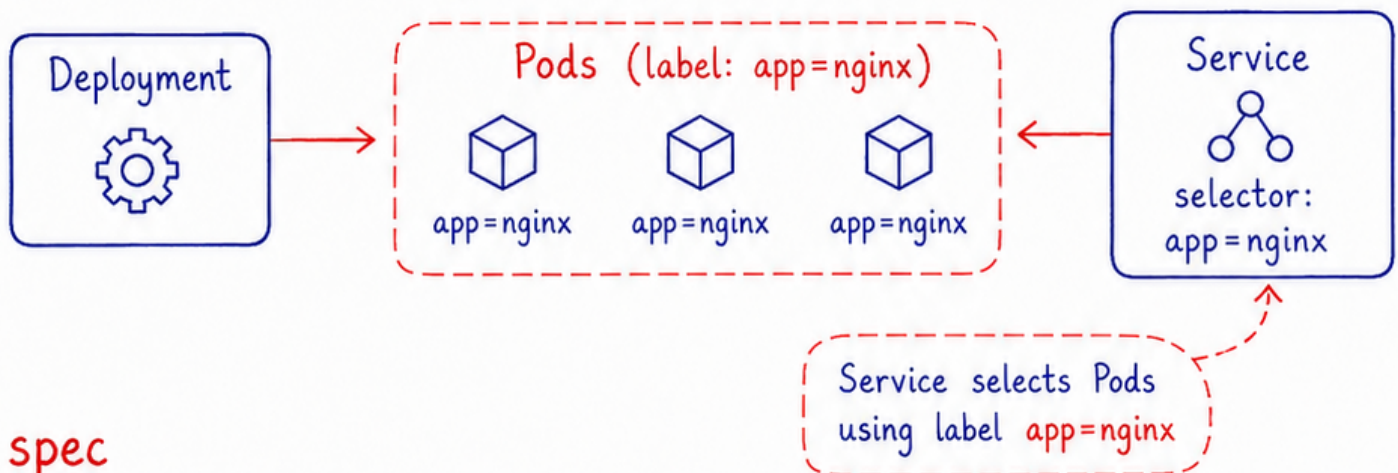
How objects are identified

- metadata contains name, namespace, labels, annotations.
- name = object name, namespace = logical separation.
- labels are simple key:value pairs.
- example labels:

- app: nginx
- environment: production



- labels help Kubernetes resources find each other.



spec

- spec = desired configuration.
- spec fields differ by resource.

Deployment spec may have:

- replicas → number of Pod copies
- selector → which Pods it manages
- template → Pod blueprint



Service spec may have:

- selector → which Pods to target
- ports → port mapping
- type → ClusterIP, NodePort, LoadBalancer, etc.



Labels + selectors are one of the most important Kubernetes concepts.



JyothiMulkuntla

Deployment YAML Explained

Jyothi Mulkuntla

From Deployment to Pods

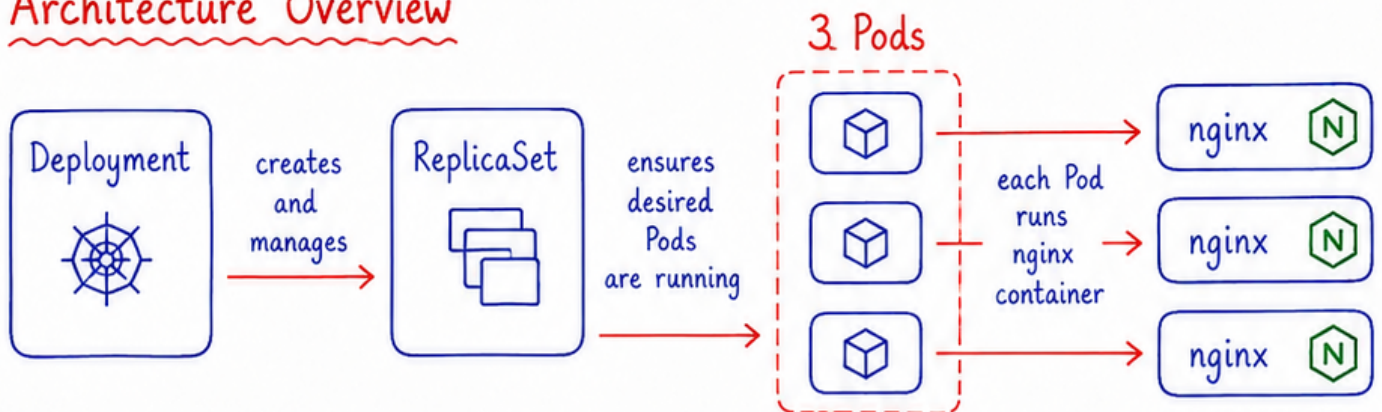
```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
          ports:
            - containerPort: 80
```

- replicas = desired number of Pods.
- selector.matchLabels = which Pods belong to the Deployment.
- template = Pod blueprint.
- containers section defines container name, image, and ports.
- the dash '-' means list item.

★ **Note:** containerPort documents the container listening port but does not expose it publicly.



Architecture Overview



★ This ensures that if a Pod fails, new Pods are created automatically to match the desired replica count.



Jyothi Mulkuntla

Service YAML Explained

Jyothi Mulkuntla

Stable network access for Pods

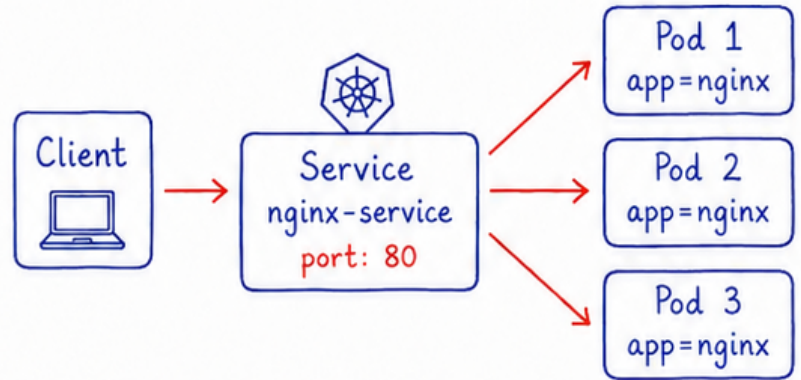
- Pods are temporary and Pod IPs can change.
- Service provides stable access to matching Pods.

Service YAML Example

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  selector:
    app: nginx
  ports:
    - port: 80
      targetPort: 80
  type: ClusterIP
```

What it means

- selector finds Pods with label **app=nginx**
- **port** = Service port
- **targetPort** = container/app port



Service types

ClusterIP



Default type. Exposes the Service on a cluster-internal IP. Only reachable inside the cluster.

NodePort



Exposes the Service on each Node's IP at a static port (30000-32767). Reachable from outside via <NodeIP>:<NodePort>.

LoadBalancer



Creates an external LoadBalancer (if supported by cloud) and exposes the Service externally.

Port vs TargetPort (Example)



★ Traffic comes to Service on **port 80**, and is forwarded to the container on **targetPort 8080**.

Quick comparison



Field	Meaning
port	Port exposed by the Service
targetPort	Port on the Pod/container that receives traffic

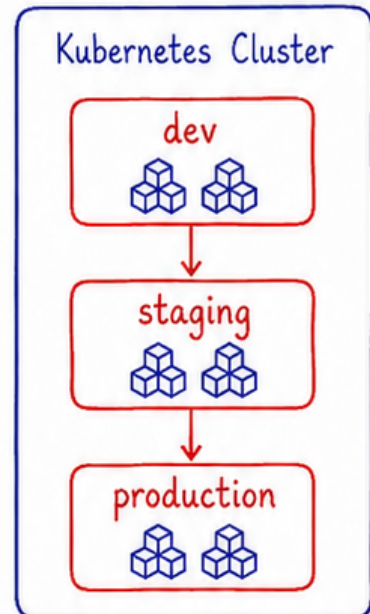
Jyothi Mulkuntla

Namespace, ConfigMap & Secret

Tyothi Mulkuntla

Configuration in Kubernetes

- Namespace separates resources logically
- Example namespaces: dev, staging, production



ConfigMap = non-sensitive configuration



Secret = sensitive data like passwords, API keys, tokens, certificates

ConfigMap data:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_ENV: production
  LOG_LEVEL: info
```

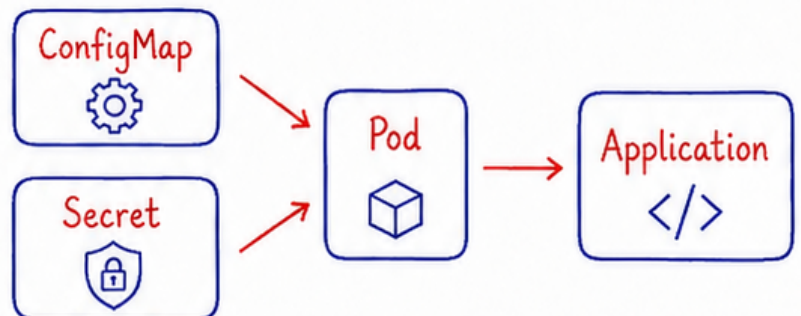
Secret stringData:

```
apiVersion: v1
kind: Secret
metadata:
  name: app-secret
  type: Opaque
stringData:
  DB_USERNAME: admin
  DB_PASSWORD: mypassword
```



Using configuration in Pods

- **env**: set individual environment variables using values from ConfigMap or Secret
- **envFrom**: load multiple key-value pairs at once from a ConfigMap or Secret



Use **ConfigMap** for normal config and **Secret** for sensitive values.



Tyothi Mulkuntla

6. Resources & Health Probes

JyothiMulkuntla

CPU, Memory and container health

resources:

requests:

cpu: "100m"

memory: "128Mi"

limits:

cpu: "500m"

memory: "512Mi"

- requests = minimum needed
- limits = maximum allowed
- 1000m = 1 CPU core
- memory units: Ki, Mi, Gi

500m = 0.5 CPU



Health Probes

☆ livenessProbe = restart container if unhealthy

☆ readinessProbe = send traffic only when ready

Liveness Probe (HTTP)

httpGet:

path: /health

port: 8080

scheme: HTTP



Readiness Probe (HTTP)

httpGet:

path: /ready

port: 8080

scheme: HTTP



Liveness



Is the container
alive and healthy?



restart?



Readiness



Is the container
ready to serve traffic?



receive traffic?



Healthy does not always mean ready for traffic.



JyothiMulkuntla

Volumes & Persistent Storage

Jyothi Mulkuntla

How Kubernetes stores data

- Container filesystem is temporary.
- Data may be lost if a Pod dies.
- Use volumes and persistent storage for important data.



Pod Spec: Using a Volume (PVC)

volumeMounts:

- name: app-storage
mountPath: /data

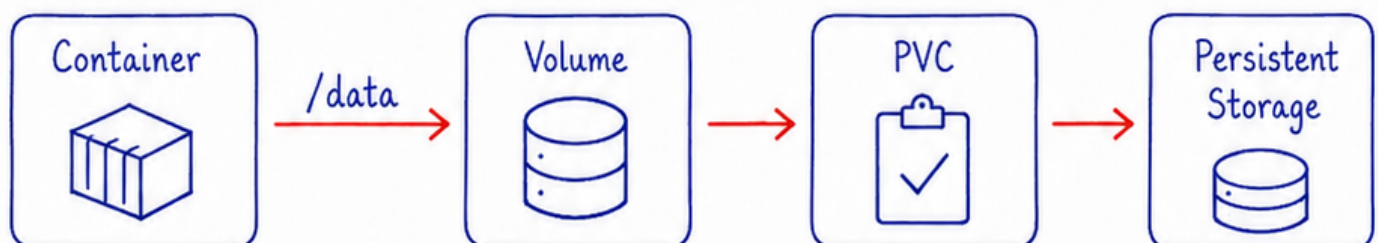
volumes:

- name: app-storage
persistentVolumeClaim:
claimName: app-pvc

PersistentVolumeClaim (PVC) Example

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: app-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
```

How it works



PVC is how the application
requests storage from Kubernetes.



Jyothi Mulkuntla

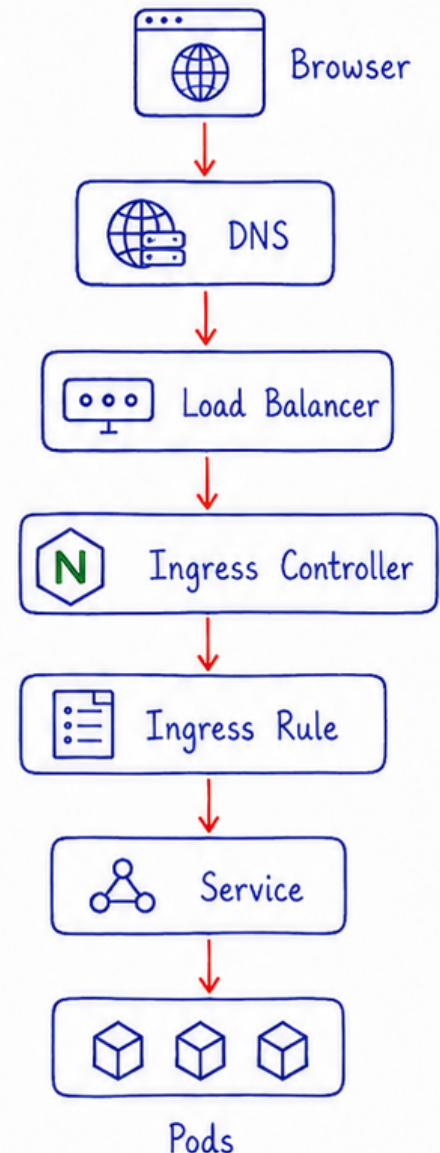
Ingress & Multi-Resource Files

Jyothi Mulkuntla

Routing HTTP/HTTPS traffic

- Service exposes apps inside Kubernetes networking
- Ingress provides HTTP/HTTPS routing rules

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app-ingress
spec:
  ingressClassName: nginx
  rules:
    - host: app.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: nginx-service
                port:
                  number: 80
```



★ Ingress object stores rules;
Ingress Controller implements them.

Multiple resources in one file

- Use YAML separator '---' to define multiple resources in a single file.

```
apiVersion: v1
kind: ConfigMap
...
---
apiVersion: apps/v1
kind: Deployment
...
```

- Apply all resources using one command:

```
> kubectl apply -f app.yaml
```

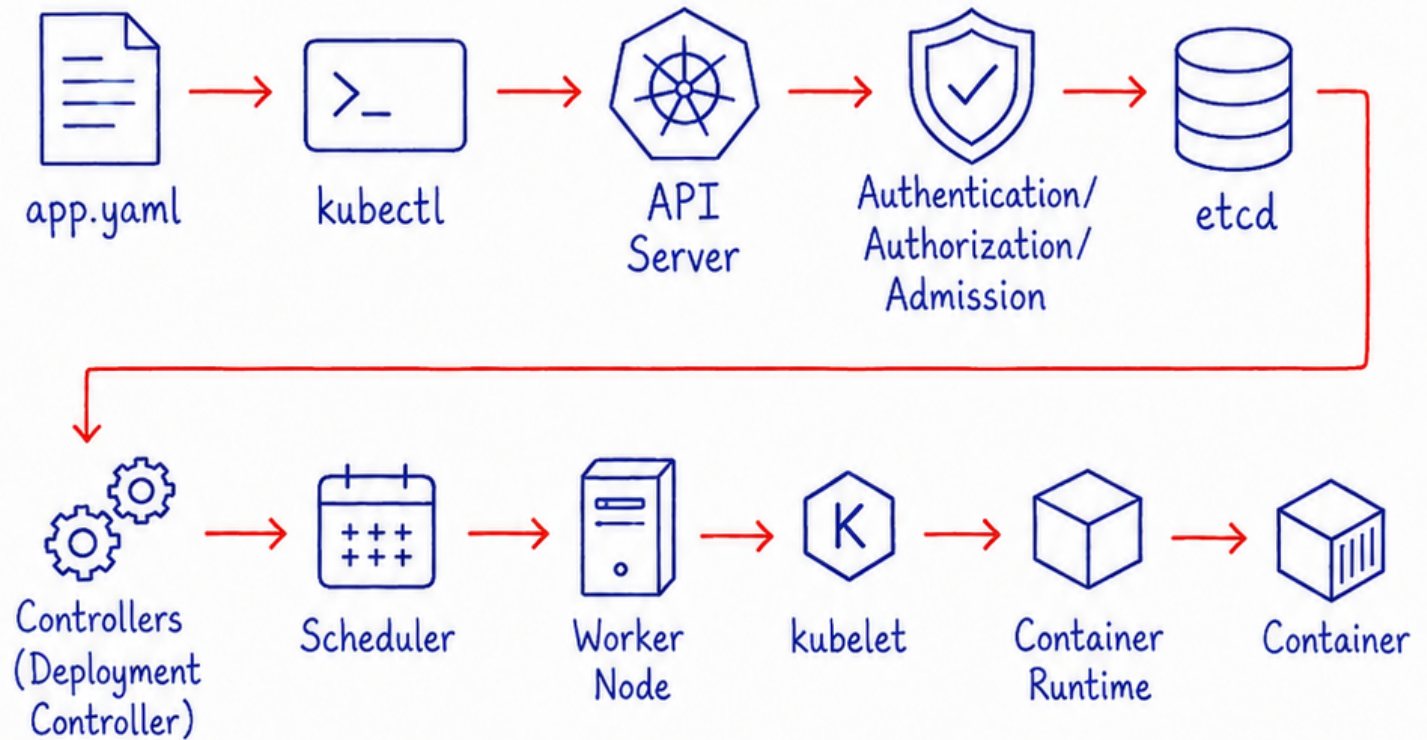
Keeps manifests organized
and easy to manage!

Jyothi Mulkuntla

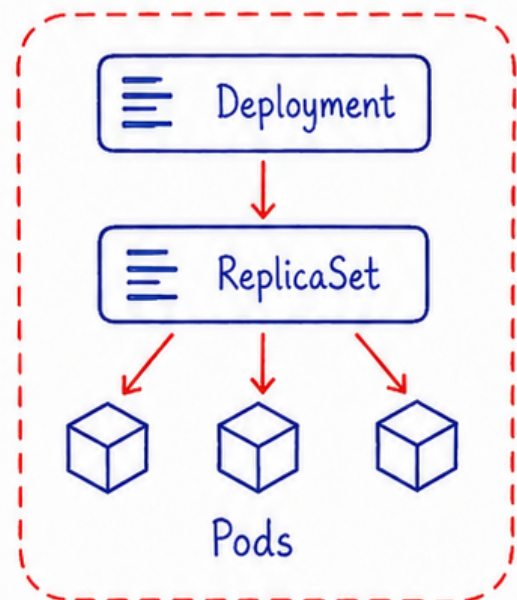
What Happens After kubectl apply?

JyothiMukkuntla

The execution flow



1. kubectl reads the YAML
2. API Server validates the request
3. desired state is stored
4. Deployment Controller creates ReplicaSet
5. ReplicaSet creates Pods
6. Scheduler places Pods on nodes
7. kubelet starts containers using the runtime
8. Service later finds Pods using labels



Kubernetes continuously works to match the desired state.



JyothiMukkuntla

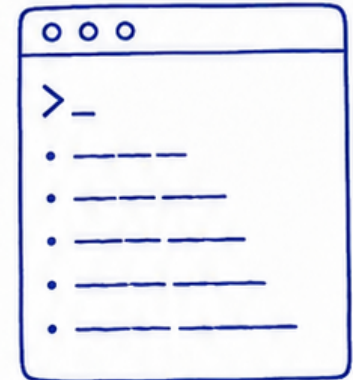
Verification, Reconciliation & Best Commands

JyothiMukkuntla

How to learn and troubleshoot

1. Useful commands:

- `kubectl get deployments`
- `kubectl get pods`
- `kubectl get svc`
- `kubectl describe deployment <name>`
- `kubectl logs <pod>`
- `kubectl get deployment <name> -o yaml`
- `kubectl explain deployment.spec`
- `kubectl create deployment nginx --image=nginx \`
`--dry-run=client -o yaml`



2. spec vs status:

- spec = desired state
- status = current/actual state

Desired replicas = 3

Ready replicas = 2

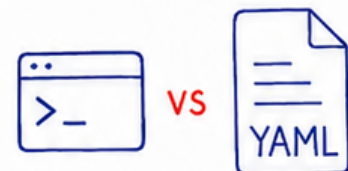
3. Reconciliation loop:



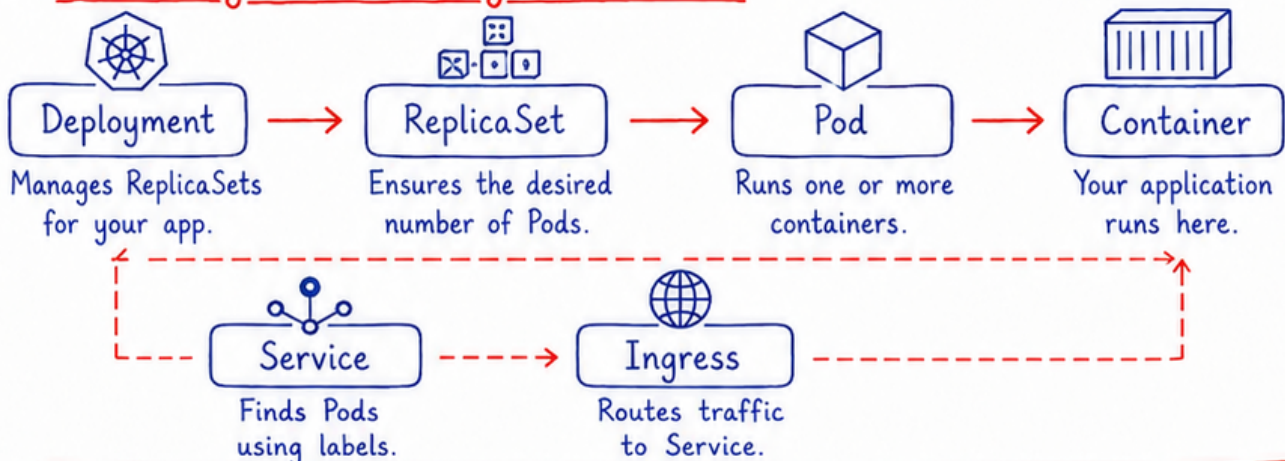
Kubernetes keeps correcting drift between desired and current state.

4. Imperative vs Declarative:

- imperative = direct command
- declarative = YAML + kubectl apply



5. How things connect (high-level flow):



★ You describe the desired state; Kubernetes reconciles it.



JyothiMukkuntla